

CSED312 Project 1

Final report

팀 오우노

20230355 정지성

20230345 이성재

목차

1. Alarm Clock

1.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

1.2. 구현방식

1.3. 논의

2. Priority Scheduling

2.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

2.2. 구현방식

2.3. 논의

3. Advanced Scheduler

3.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

3.2. 구현방식

3.3. 논의

1. Alarm Clock

1.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

기존의 핀토스 구현은 busy waiting 방식으로, sleep 해야 하는 스레드도 ready list에 들어가 있어서 cpu가 계속 해당 스레드를 확인해야 하는 비효율이 있었다. 우리가 구현한 알람 클럭은 `timer_sleep()`이 실행되면 일정 시간 동안 스레드를 block 시켜 이러한 비효율을 피할 수 있다.

1.2. 구현방식

1.2.1. 자료구조

스레드가 block 되기 전, 해당 스레드가 일어날 시간을 저장해야 한다. 이는 스레드에 종속적이어야 하므로 thread 구조체에 `tick_to_wake` 라는 인스턴스를 추가했다. 또한 `timer.c` 에는 잠들어 있는 스레드들을 관리하기 위한 `sleeping_list`를 추가했다. 이 list를 이용하기 위해 thread에 `sleep_elem` 이라는 `list_elem` 변수도 추가하였다.

1.2.2. 알고리즘

Alarm clock을 구현하기 위해서는 `timer_sleep()` 함수를 수정해야 했다. 이 함수가 호출된 시점의 tick을 먼저 저장하고, 여기에 매개변수로 들어온, 즉 자야하는 tick 시간을 더해서 현재 스레드의 `tick_to_wake` 변수에 저장한다. 그 다음으로 `sleeping_list`에 해당 스레드의 `sleep_elem`을 푸쉬하고, 해당 리스트를 `tick_to_wake`를 기준으로 정렬한다. 이 리스트를 정렬하지 않으면 매 tick마다 모든 원소들이 일어날 시간인지 검사해야 하지만, 미리 정렬을 해두면 가장 앞의 원소만 검사하면 되므로 효율적이다. 이 비교를 위해서 `compare_wake_time()` 이라는 함수를 만들어야 했다. 일어날 시간을 저장하고, `sleeping_list`에 자신을 추가했으니 실제로 `thread_block()`을 통해 스레드를 block한다. 이 모든 과정들은 인터럽트를 비활성화 하고 실행되어야 한다.

1.3. 논의

1.3.1 Design report와 달라진 점

처음 디자인을 할 때는 `sleeping_list`를 정렬하지 않고 매 tick마다 원소들을 순회해도 큰 손해가 아닐 거라고 생각했지만, 구현을 할 때는 정렬을 하는 것으로 결정했다. Tick에서 시간이 오래 걸리면 테스트 케이스가 실패할 수도 있다는 사실을 알게 되었기 때문이다.

1.3.2 배운점

앞으로 계속 봐야할 핀토스 코드에 조금이나마 익숙해질 수 있었다. 특히 list를 다루는 방식에 익숙해질 수 있었던 것 같다. 평소에 쓰는 흔한 list가 아니라 doubly linked list 자료구조를 사용해서 모든 구현을 해놓았기 때문에 이에 적응하는 것이 도움이 됐다.

1.3.3 한계점

Sleeping_list에 접근할 때 인터럽트를 비활성화 하는데, 작업의 특성상 꽤 오랜 시간 인터럽트가 비활성화 될 수 있다. Sleeping_list가 커지면 전체적인 tick 처리 속도가 느려질 수 있다.

2. Priority Scheduling

2.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

- 기존의 pintos는 priority를 서포트하지 않아서, 모든 thread가 공평하게 round robin 방식으로 schedule되고 semaphore나 lock, conditional variable 등도 기다린 순서대로 차례를 받았었다.
- Project 1의 2번 과제를 통해서 Pintos에 priority를 도입하였다. 특히, priority inversion 문제를 해결하기 위해 priority donation 방식을 활용했다. 이를 통해 os가 더 중요한 thread를 먼저 실행시켜 사용자 이용성 등을 향상시킬 수 있다.

2.2. 구현방식

2.2.1. 계획

- 우선 두 개의 step으로 나눠서, 각각의 behavior을 최대한 잘 정의했다.
- (a) Priority donation. Donation은 lock을 기다리는 상태일 때, effective_priority를 위로(= lock holder으로) 올리는 행위를 말한다.
- Donation의 생성과 업데이트는 lock을 기다리기 시작했거나, donation을 받았으며 자신의 effective_priority가 증가했을 경우 행하면 된다. 구체적으로, lock holder이 존재함을 확인한 뒤 lock holder에게 donation을 한 후, lock holder이 속한 list (ready list, wait list)에 sort되게끔 넣어주면 된다. 이 때, “donation”은 사실 그저 effective priority를 다시 계산해주는 행위로 실행할 수 있다. effective priority의 계산은 현재 thread가 가진 모든 lock들의 wait list들의 effective priority와 자신의 base priority의 maximum을 계산하는 방식으로 구현했기 때문에 가능하다.
- Donation의 삭제는 thread가 lock을 release했을 때 행하면 된다. 이 때 lock releaser의 effective priority를 다시 계산해 주고 thread가 속한 list에 sort되게끔 넣어주면 된다.
- (b) Sorting lists & immediately yielding. Donation 외에는, Ready list와 wait list를 상시 정렬된 상태로 유지하고, ready list에서 running thread보다 큰 priority가 발생했을 때 즉시 yielding하는 것만 신경써주면 된다.
- sorting은 add to ready list, add to wait list, modify priority의 세 경우로 나누어 생각할 수 있다. Add to ready list는 `thread_unblock()`, `thread_yield()`의 경우에 일어난다. Add to wait list는 `sema_down()`, `lock_acquire()`, `cond_wait()`의 경우에 일어난다. (우리의 implementation에서 `lock_acquire()`는 `sema_down()`과 분리했으므로 따로 고려해야 한다.) Modify priority는 `thread_set_priority()`와 priority donation의 경우에서 고려해야 한다. (코드를 보면 알겠지만 priority donation은 `thread_update_effective_priority()`에서 모두 처리하고, `thread_set_priority()`에서는 위의 함수를 불러오기 때문에 `thread_update_effective_priority()`에서만 list의 sorting ensurance를 해주면 된다.)

- Immediately yielding은 `thread_unblock()` 이후의 코드들, 즉 `sema_up()`, `lock_release()`, `thread_create()` 등등과, priority를 낮게 업데이트할 수 있는 `thread_set_priority()`에서 한번씩만 체크해 주면 된다.

2.2.2. functions and data structures

Data structures: struct thread에 `int effective_priority`, `struct lock *waiting_lock`, `struct semaphore *waiting_sema`, `struct list *locks_holding`을 추가로 정의했다. 각각이 사용되는 곳, 업데이트되는 곳을 정리하면 다음과 같다.

- `effective_priority`: 새로 정의한 함수인 `thread_update_effective_priority()`에서 업데이트되며, priority를 이용해 비교해야 하는 모든 곳에 사용된다. `int priority`를 그대로 쓰지 않는 이유는 모든 donation이 사라지면 base priority값으로 돌아가야 하기 때문이다.
- `waiting_lock`: 현재 thread가 lock에 의해 block되고 있으면 그 lock의 pointer을, 그렇지 않으면 `NULL`을 저장하는 값이다. `thread_update_effective_priority()`에서 thread의 wait list를 업데이트해야 할 때 사용하고, upward donation을 해야 하는지 확인할 때도 사용한다. 이는 `lock_acquire()` 함수에서 block 전에 설정하고, block 이후에 바로 다시 `NULL`을 저장하면 된다.
- `waiting_sema`: 현재 thread가 semaphore에 의해 block되고 있으면 그 semaphore의 pointer을, 그렇지 않으면 `NULL`을 저장하는 값이다. 이는 `sema_down()` 함수에서 block 전에 설정하고, block 이후에 바로 다시 `NULL`을 저장하면 된다.

*** 참고로 conditional variable의 정렬을 위해서 thread에 추가적 구조를 부여하지는 않아도 된다. 대신 condvar 자체가 유지하는 semaphore list element에 thread정보를 추가해, thread의 priority 순서대로 리스트를 정렬했다.

- `locks_holding`: thread가 hold하고 있는 lock들을 저장한 list이다. 이는 `thread_update_effective_priority()` 함수에서 실제 effective priority를 계산할 때 donor들을 알아보는 용도로 사용된다. (각 lock의 wait list의 앞을 보면 그 lock에서 donate받은 값을 알 수 있다. Wait list는 우리가 수시로 sort하기 때문이다.) 업데이트는 `lock_acquire()`와 `lock_release()`에서 각각 list에 추가하고 삭제하면 될 것이다. (`lock_try_acquire()`에서도 추가해야 할 것이다.)

Functions: 가장 중요한 function은 `thread_update_effective_priority()`이다. 그 외에도 helper function인 `yield_if_we_should()`과 `compare_effective_priority()` 등이 있다. 앞서 정의한 `struct thread`의 새로운 element들을 잘 초기화하기 위해 위에 명시된 함수들을 수정해야 하기도 했다. 마지막으로, conditional variable의 priority 순서를 위해 `compare_waiter_priority()`를 정의했으며 이에 따라 `cond_wait()`도 알맞게 수정해주기도 했다.

다음으로, 제일 중요하게 추가된 세 함수 `thread_update_effective_priority()`, `compare_effective_priority()`, `yield_if_we_should()`에 대해 설명하겠다.

- `thread_update_effective_priority()`

이 함수는 `struct thread` 포인터 하나를 받아, 그 thread의 effective priority를 업데이트하는 기능을 한다. 이 과정에서 각종 list의 정렬 유지, lock holder에게 priority 넘겨주기(donate) 등의 기능을 함께 수행한다. 함수의 기능을 실행 순서로 정렬하면 다음과 같다.

1. Argument로 받은 thread `t`의 effective priority 갱신. 이 과정에서 `t`의 `locks_holding` 리스트와, 각 lock의 wait list를 모두 참조하므로, `synch.c`의 다른 코드들에서 위의 값들을 아주 잘 갱신시키는 것을 전제로 한다.
2. `t`의 priority가 실제로 바뀌었으며 `t`가 ready list나 wait list에 있었다면, `t`가 속해있던 list를 재정렬한다. `t`의 priority만 업데이트된 것이므로, 이 기능은 `t`를 해당 리스트에서 제거했다가 다시 추가하는 방식으로 구현할 수 있다.
3. `t`가 기다리는 lock이 있다면, 그 lock의 holder의 effective priority도 업데이트시킨다. 이 과정이 바로 priority donation이다.

또, 이 함수가 사용되는 곳들을 모두 모아 정리하면 다음과 같다.

1. 재귀적: priority donation의 과정에 사용된다. 이 때, loop에 갇히지 않기 위해, donation은 실제 running thread의 effective priority가 변화되었을 때만 진행한다.
2. `thread_set_priority()`: 이 함수는 running thread의 **base priority**를 업데이트시킨다. 이에 따라 effective priority가 바뀔 수도 있으므로, 해당 함수를 사용한다.
3. `lock_acquire()`: lock을 기다릴 때, priority를 donate하기 위해 해당 함수를 사용해 lock holder의 effective priority를 업데이트시킨다.
4. `lock_release()`: lock을 release하면 donate 받았을 수도 있는 priority를 떼어내야 한다. 그러기 위해서 먼저 lock의 wait list와 running thread의 `locks_holding()`을 잘 초기화한 후 해당 함수를 사용해 알맞게 donated priority를 떼어낸다.

- `compare_effective_priority()`

Ready list나 wait list에서 정렬한 채로 넣고 싶을 때 사용하는 helper function이다. 두 thread `list_elem`을 받아, 앞의 thread의 priority가 더 높으면 true를 반환하고 그렇지 않으면 false를 반환한다. 한 가지 눈여겨볼 것은 tie가 일어났을 때 false를 반환한다는 것인데, 이는 의도적인 선택이다. 이 선택에 의해 list들이 round robin 형태로 구현된다. (이렇게 하면 같은 priority끼리 비교할 때 비교대상을 더 작은 것 취급해서, 같은 priority element 중 가장 뒤쪽으로 배치시킬 수 있다.)

- `yield_if_we_should()`

Ready list의 top element와 running thread의 priority를 비교해서, running thread의 것이 더 낮으면 yield를 하는, 꽤나 간단한 기능을 하는 함수이다. 이 함수가 사용되는 곳들을 모두 모아 정리하면 다음과 같다.

1. `thread_unblock()` 이후

- `thread_create()`: thread creation의 마지막 단계는 방금 생성한 thread를 unblock하는 것이다. 이 직후에 yield해야 한다면 하는 것이 맞으므로, 해당 함수를 불러온다.
- `sema_up()`: semaphore의 value를 증가시킨 뒤, 기다리는 thread가 있으면 하나를 unblock하는 로직이 있다. 이 직후에 yield를 해야 한다면 하는 것이 맞으므로 해당 함수를 불러온다. 참고로 lock과 conditional variable의 unblock도 `sema_up()`을 사용하므로 모든 synchronization primitive의 unblock 케이스에 대한 커버를 완료했다.

*** 참고로, 처음에는 `thread_unblock()` 함수의 말미에 해당 함수를 불러올까도 생각했는데, 이렇게 하면 무슨 일에서인지 프로그램이 동작하지 않아서, 코드 내에 `thread_unblock()`이 있을 때마다 직후에 해당 함수를 불러오는 식으로 코드를 짰다.

2. 더 낮은 effective priority를 얻은 이후

- `thread_set_priority()`: donate받은 priority들보다 base priority가 더 컸던 상태에서 base priority가 감소하면 더 낮은 effective priority를 얻게 된다. 따라서 이때 `yield_if_we_should()`를 불러와 한 번 체크해 주어야 한다.
- `lock_release()`: lock을 release하고 나면 donate받은 priority가 날라가서 더 낮은 effective priority를 얻게 될 수 있다. 이때에도 해당 함수를 불러 한 번 체크해야 한다.

*** 지금 생각해보니 `thread_update_effective_priority()` 함수의 말미에 해당 함수를 불러올 수도 있을 것 같다. 처음 코드를 짤 때에는 `thread_update_effective_priority()`는 재귀적인 함수여서, 원치않는 위험을 줄이기 위해 함수에서 분리시켰다.

2.3. 논의

2.3.1. 디자인 레포트와 비교해 달라진 점

- `struct thread`에 `waiting_lock`과 `waiting_sema`를 추가해 주었다. 디자인 레포트를 작성할 때에는 이런 것들 없이도 thread가 속해 있는 list를 쉽게 얻어낼 수 있을 줄 알았는데, Pintos에서 list element는 자신이 속한 list가 어디에 있는지 쉽게 찾아내는 방법이 없다는 것을 코딩을 하며 깨달았으며, 그 때문에 두 structure을 추가해 줬다. 또한 `waiting_lock`이 있어야 `thread_update_effective_priority()`에서 'donor'들을 따로 저장하지 않고도 불러올 수 있어서, 그런 방식으로도 활용할 수 있었다.
- `thread_set_priority()`와 `thread_update_effective_priority()`를 원래는 구별된 로직으로 짜려고 그랬는데, 생각해 보니 `thread_set_priority()`에서는 base priority만 업데이트시키고 바로 `thread_update_effective_priority()`를 불러오면 된다는 것을 깨달았다.
- 그 외의 high level에서의 컨셉은 디자인 레포트의 것을 거의 따랐다.

2.3.2. 배운 점

- Context switching에 대해 더 잘 이해하게 되었다. Context switching을 이렇게나 깊게 고려해야 하는 코딩은 살면서 처음 해보는 것 같다. 그만큼 엄청 헷갈렸는데, 그래도 완성된 코드를 보고 나니 뿌듯했다. 앞으로 나오는 pintos 프로젝트들에 더 잘 적응할 수 있을 것 같다.

2.3.3. 한계점

- Mlfqs와 완전한 호환이 되지는 않는다. Pintos manual에서 2번과 3번을 동시에 테스트하지 않는다는 것을 참고하여 내린 디자인 결정인데, 그래도 한계점이라면 한계점이다.
- Starvation이 일어날 수 있다. 로직에서 priority update가 거의 static한 것이 큰 이유로 작용한다. 좋은 dynamic scheduler 알고리즘을 사용하면 어느 정도 해결할 수는 있을 것 같다.

3. Advanced Scheduler – 4.4BSD

3.1. 목표 (기존의 Pintos와 비교해서 나아진 것)

기존 핀토스 코드에는 advanced scheduler가 구현되어 있지 않았다. 2번을 구현함으로써 priority를 기반으로 스케줄링을 할 수 있게 되었지만 코드 자체적으로 우선순위를 유동적으로 변경할 수는 없었다. 반면 4.4BSD를 구현하면 각 스레드가 cpu 사용시간, 현재 load 등을 기준으로 priority를 늘리거나 줄일 수 있다. 이는 더 효율적인 cpu 사용을 가능케한다.

3.2. 구현방식

3.2.1. 자료구조

각 스레드는 nice 값과 `recent_cpu` 값을 인스턴스로 가져야 한다. 또한 `thread.c`에 전역변수로 `load_avg`를 선언했다. 이 세개의 변수들을 이용해서 priority를 결정하게 된다. MLFQS를 위한 ready list도 별도로 만들어 줬는데, 이를 `mlfqs_ready_lists`라고 했다. 이는 `PRI_MAX + 1`개의 배열을 원소로 하는 리스트로 인덱스를 이용해 특정 priority 큐에 접근할 수 있다. 예를 들어 `mlfqs_ready_lists[30]`은 priority가 30인 스레드들이 들어있는 큐를 의미하는 것이다.

3.2.2. 알고리즘

가장 먼저 구현해야 할 것은 fixed point 연산을 위한 로직이었다. 이는 `fixed_point.h` 라는 파일을 만들어 구현했으며 `F`를 ($1 < F < 14$)로 정의해 14.17 fixed point number representation을 사용하도록 했다. 내부에 구현한 함수는 총 11개로, pintos 문서에 있는 표를 참고했다.

```
int int_to_fp (int n);
int fp_to_int (int x);
int fp_to_int_round (int x);
int add_fp (int x, int y);
int sub_fp (int x, int y);
int add_fp_int (int x, int n);
int sub_fp_int (int x, int n);
int mult_fp (int x, int y);
int mult_fp_int (int x, int y);
int div_fp (int x, int y);
int div_fp_int (int x, int n);
```

- thread_tick()

thread_tick()에서는 적절한 타이밍에 load_avg, recent_cpu, 그리고 priority 를 바꿔주는 함수를 호출해야 한다. 이는 thread_mlfqs가 true인 경우에만 처리 되어야 한다.

```
if (thread_mlfqs){
    if (t != idle_thread){ // Ignore idle thread
        t->recent_cpu += int_to_fp(1);
    }

    if(timer_ticks() % TIMER_FREQ== 0){ // Every second
        mlfqs_update_load_avg();
        mlfqs_update_all_recent_cpu();
    }

    if(timer_ticks() % 4 == 0){ // Every 4 ticks
        mlfqs_update_all_priority();
    }
}
```

각 조건문은 역시 pintos 문서의 appendix B를 참고해 작성했다. load_avg를 업데이트 하는 식은 다음과 같이 작성했다.

```
load_avg = add_fp( mult_fp(div_fp_int(int_to_fp(59), 60), load_avg),
                    div_fp_int(int_to_fp(ready_threads), 60) );
```

두번째 항을 보면 문서에는 (1/60)을 ready_threads에 곱하는 식으로 계산하지만 구현에는 ready_threads에 60을 나누는 식으로 작성되어 있음을 알 수 있다. ready_threads는 mlfqs_ready_lists에 포함된 모든 스레드의 개수 + 현재 실행중인 스레드의 개수를 의미한다.

```
for (int i = PRI_MIN; i <= PRI_MAX; i++){
    ready_threads += list_size(&mlfqs_ready_lists[i]);
}
```

```
if (thread_current() != idle_thread) ready_threads++;
```

아래의 코드는 idle_thread를 제외하고 ready_threads를 세어야 하기 때문에 필요하다. 현재 실행 중인 스레드가 idle thread 이면 제외해야 하고, 그 외의 경우 실행 중인 스레드 개수(즉, 1개)를 더해줘야 한다. mlfqs_update_all_recent_cpu()는 모든 프로세스를 순회하며 mlfqs_update_recent_cpu()를 호출하는 함수로, mlfqs_update_recent_cpu()에서는

$$recent_cpu = (2 * load_avg) / (2 * load_avg + 1) * recent_cpu + nice.$$

식을 바탕으로 `recent_cpu`를 업데이트 한다. 위의 두 함수는 1초마다 실행되는 함수임을 기억하자. 반면 `mlfqs_update_all_priorit()`는 4-tick마다 실행되는 함수로, 모든 스레드를 순회하며 `mlfqs_calculate_priority()`를 호출하는 함수이다. 이 함수에서는

$$\text{priority} = \text{PRI_MAX} - (\text{recent_cpu} / 4) - (\text{nice} * 2)$$

식을 바탕으로 `priority`를 업데이트 한다. 이때 계산된 식이 `PRI_MIN`보다 작으면 `PRI_MIN`으로, `PRI_MAX`보다 크면 `PRI_MAX`로 `priority`를 설정해준다.

- `thread_create()`

이 함수에서는 자식 스레드가 부모 스레드의 `nice`값과 `recent_cpu` 값을 물려받도록 로직을 추가했다.

```
if(thread_mlfqs){
    t->nice = thread_current()->nice;
    t->recent_cpu = thread_current()->recent_cpu;
}
```

- `thread_unblock()` & `thread_yield()`

스레드가 `unblock` 되면 해당 스레드를 `ready list`에 올려야 한다. `Mlfqs`에서는 `mlfqs_ready_lists`에 넣어야 하므로 해당 로직을 추가했다.

```
if (thread_mlfqs){
    list_push_back(&mlfqs_ready_lists[t->priority], &t->elem);
}
else{
    list_insert_ordered (&ready_list, &t->elem, compare_effective_priority, NULL);
}
```

- `yield_if_we_should()`

이 함수에서는 스레드가 `yield`를 해야 하는지 여부를 판단해야 한다. `thread_mlfqs`가 켜진 상태에서는 다른 로직을 사용해야 하며, 아래 코드가 핵심이다.

```
for (int i = PRI_MAX; i > cur->priority; i--) {
    if (!list_empty(&mlfqs_ready_lists[i])) {
        should_yield = true;
        break;
    }
}
```

이 코드는 현재 실행중인 스레드보다 우선순위가 높은 큐를 순회하며 원소가 하나라도 있을 경우 `should_yield`를 `true`로 바꾸어 더 높은 우선순위의 스레드에게 양보해야 함을 알린다.

```

if (should_yield) {
    if (intr_context()) {
        intr_yield_on_return();
    } else {
        thread_yield();
    }
}
}

```

이후 실제로 yield를 수행하며, 이 과정은 우선순위가 바뀌는 즉시 yield를 해야 통과하는 mlfqs-block 테스트를 통과하는 데에 필요했다.

- thread_set_nice()

매개변수로 받은 값으로 thread의 nice 값을 바꾸고, 그 즉시 다시 priority를 계산한다. 다시 계산한 priority를 바탕으로 자신이 최우선 스레드가 아님을 알게되면 그 즉시 thread_yield()를 호출해서 주도권을 다른 스레드에 넘긴다.

```

mlfqs_calculate_priority(current_thread);
// todo: yield if necessary
for (int i = PRI_MAX; i > current_thread->priority; i--){
    if (!list_empty(&mlfqs_ready_lists[i])){
        thread_yield();
        break;
    }
}
}

```

그리고 스케레톤 코드로 주어졌던 thread_get_nice(), thread_get_load_avg(), thread_get_recent_cpu() 등의 함수도 적절히 작성했다.

- next_thread_to_run()

이 함수에서는 mlfqs_ready_lists를 우선순위가 높은 큐부터 순회하며 가장 먼저 만나는 thread를 반환하도록 반복문을 작성했다. 또한 반환되는 스레드는 mlfqs_ready_lists에서 삭제 되도록 list_pop_front()을 이용했다.

3.3. 논의

3.3.1. Design report와 달라진 점

4.4BSD를 구현하기 위해 필요한 변수들, 즉 `load_avg`, `recent_cpu` 등을 구현할 때 사용한 수식 자체는 디자인 레포트와 달라진 부분이 없었다. 다만 디자인 단계에서 미처 고려하지 못한 부분들이 있었다.

`thread_set_nice()`에는 단순히 식 계산 뿐만 아니라 `yield`를 해야할지 여부를 판단하는 코드도 필요했고, 사용한 자료구조도 일반적인 priority scheduling에서 사용하는 list와 달라야 했다.

3.3.2. 배운 점

스레드의 우선순위를 이렇게 유동적으로 바꾸면서 스케줄링을 할 수 있다는 사실을 알게 되었다. 또한 정수 연산을 이용해서 부동소수점 연산을 구현하는 방법도 알게 되었다. 무엇보다도 스레드의 스케줄링이 이뤄지는 방식에 대해 깊은 이해를 할 수 있었다.

3.3.3. 한계점

우리의 구현 방식에서는 `mlfqs_ready_lists`를 선언하는데, 이는 64개의 큐를 유지하는 것이다. 실제로는 대부분의 큐가 비어 있더라도 메모리를 차지 하고 있다는 부분이 아쉬운 부분이다. 또한 근본적으로 priority를 업데이트 하는 수식에 포함된 상수가 pintos 문서에도 밝혀져 있듯이 경험적으로 잘 작동하는 것으로 알려진 수일 뿐, 어떤 수학적/이론적 근거가 있는 수가 아니라는 점도 해당 구현의 아름다움을 감소시키는 요인이라고 생각한다.